Java 集合面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

ConcurrentHashMap 和 Hashtable 的区别是什么？

ConcurrentHashMap 和 Hashtable 都是线程安全的哈希表实现，但它们的并发控制策略完全不同。

Hashtable 用的是方法级别的 `synchronized` ，整个对象一把锁，任一时刻只有一个线程能访问。比如 `get` 、`put` 都是同步方法，竞争激烈时性能很差，10 个线程只能串行执行。

ConcurrentHashMap 在 JDK 8 后采用 **CAS + synchronized** 的方式，只锁链表或红黑树的头节点，锁粒度小得多。默认分成了 16 个段（桶），不同段之间操作互不干扰，写操作并发度能到 16 以上。

```
// put 操作不会阻塞读
map.put("key", "value");
// 读操作几乎无锁，利用 volatile 保证可见性
String val = map.get("key");
```

Hashtable 已经基本被淘汰，除了遗留系统外不建议使用。高并发场景下 ConcurrentHashMap 的吞吐量能高出一个数量级，像 Dubbo、RocketMQ 这些中间件里大量使用 ConcurrentHashMap 来缓存元数据或管理连接。

还有一个细节：ConcurrentHashMap 不允许 key 或 value 为 null，避免在判断 `containsKey` 和 `get` 返回 null 时产生歧义。而 Hashtable 虽然允许 null 值，但实际用起来容易出 NPE，也不推荐。

Java 中的 List 接口有哪些实现类？

Java 里的 List 接口最常用的实现就三个：ArrayList、LinkedList，还有一个 CopyOnWriteArrayList。

ArrayList 底层是数组，随机访问特别快，get 和 set 基本是 O(1)，但中间插入或删除元素得搬数据，平均是 O(n)。它不是线程安全的，多线程写会出问题。平时用得最多，毕竟查得多改得少的场景太常见了，比如从数据库捞出一坨数据遍历展示。

LinkedList 是双向链表，每个节点存前后指针。它的优势在头尾增删，addFirst、addLast 都是 O(1)，但随机访问要从头/尾一路找，get 操作是 O(n)。内存开销也大点，每个节点多两个引用。一般用得少，除非明确要在两端频繁操作，比如做队列，可以用它实现 Deque。

1）ArrayList：数组实现，查询快，增删慢（尤其中间），非线程安全
2）LinkedList：双向链表，头尾增删快，查询慢，内存占用高
3）CopyOnWriteArrayList：写时复制，读不加锁，写时复制整个数组再替换，适合读多写少的并发场景，比如监听器列表

代码上，平时声明都用 List 接口：

```
List<String> list = new ArrayList<>();
list.add("hello");
```

CopyOnWriteArrayList 虽然线程安全，但写操作代价高，大列表搞不定。如果只是单线程或者有外部同步，直接用 ArrayList 就行。

Java 中 ArrayList 和 LinkedList 有什么区别？

ArrayList 和 LinkedList 都是 List 接口的实现，但底层结构完全不同，带来的性能特征也差很多。

1）ArrayList 内部用数组存储元素，查询快，增删慢。因为数组在内存中是连续的，通过下标访问能做到 O(1)。但插入或删除元素时，后面的所有元素都得往前或往后挪，平均时间复杂度是 O(n)。而且数组有容量限制，满了就得扩容，一般是 1.5 倍扩容，会触发一次复制操作。

2）LinkedList 是双向链表，每个节点存前驱和后继指针。它的优势是插入和删除只改指针，只要找到位置就能 O(1) 完成。但查找就得从头或尾开始遍历，最坏要走 n/2 步，所以随机访问慢。

一般情况下，90% 的场景都该用 ArrayList。除非你的业务逻辑频繁在中间插入删除，比如维护一个实时变动很大的队列，而且很少按索引查，才考虑 LinkedList。

代码上看区别也很明显：

```
List<String> list = new ArrayList<>();
list.add("a"); // 数组末尾加，基本不涉及移动

List<String> linked = new LinkedList<>();
((LinkedList<String>) linked).addFirst("head"); // 头插，链表擅长的操作
```

别被“链表插入快”误导了，前提是“已经定位到位置”。如果还要先遍历找位置，那整体还是 O(n)。真正高效的是在已知节点附近操作，比如迭代过程中用 ListIterator。

数据结构对比

Java 中的 HashSet 和 HashMap 有什么区别？

HashSet 和 HashMap 虽然都是基于哈希表实现的集合类，但它们解决的问题不一样。

HashMap 是存键值对的，你给一个 key，它帮你关联一个 value。查找、插入、删除基本都在 O(1) 时间完成，前提是哈希分布均匀。它的底层是数组 + 链表（或红黑树），冲突多了链表长度超过 8 就转成红黑树，提升查找性能。

HashSet 其实就是披着集合外衣的 HashMap。它内部**持有一个 HashMap 实例**，存元素时，把元素作为 key，统一指向同一个静态 Object 对象作为 value。所以 HashSet 只能存不重复的元素，本质上是利用了 HashMap 的 key 不可重复特性。

1）HashMap 允许存放 null 键和 null 值，但最多一个 null 键。 2）HashSet 也允许一个 null 元素，原因同上。 3）遍历顺序不一定，除非用 LinkedHashMap 或 LinkedHashSet。

代码上看，HashSet 的 add 方法其实就是调用了 map.put(e, PRESENT) == null 来判断是否新增成功。

一般你要存映射关系就用 HashMap，只关心去重和存在性判断就用 HashSet，背后的脏活累活其实都是 HashMap 在扛。

Java ArrayList 的扩容机制是什么？

ArrayList 的扩容发生在添加元素时容量不足的情况下，整个过程其实就几步：先检查是否需要扩容，再按比例增长，最后复制数据。

1） 添加元素前会判断当前 size + 1 是否超过数组长度
2） 如果超过，就会触发 grow() 方法进行扩容
3） 新容量是旧容量的 1.5 倍，计算方式是 `oldCapacity + (oldCapacity >> 1)`
4） 扩容后通过 System.arraycopy 将原数组数据拷贝到新数组

初始容量默认是 10。第一次 add 时如果没指定大小，会创建一个长度为 10 的数组。后续每次扩容都按 1.5 倍往上加，比如从 10 到 15，再到 22、33……直到 Integer.MAX_VALUE。

```
private void grow() {
    int oldCapacity = elementData.length;
    int newCapacity = oldCapacity + (oldCapacity >> 1);
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

频繁扩容会影响性能，因为每次都要内存分配和数组拷贝。如果能预估数据量，建议初始化时直接指定容量，比如 `new ArrayList<>(1000)` ，这样能避免中间多次扩容的开销。

扩容本身是**脏活累活**，但对上层透明。你只管 add，它自动帮你扛住容量问题，前提是别在循环里无脑 add 大量数据而不预设容量。

什么是 Hash 碰撞？怎么解决哈希碰撞？

哈希碰撞指的是不同的输入数据经过哈希函数计算后，得到了相同的哈希值。这种情况在实际使用中几乎是不可避免的，毕竟哈希空间有限，而输入可能是无限的。

比如用 `HashMap` 存储键值对时，两个不同的 key 算出来的 hash 值一样，就会发生碰撞。这时候如果不处理，数据就会被覆盖或者丢失。

解决哈希碰撞主要有两种常见方式：

1）链地址法（Separate Chaining）
每个哈希桶对应一个链表或红黑树，所有哈希到同一位置的元素都存放在这个结构里。Java 的 `HashMap` 就是这么干的。当链表长度超过 8 且数组长度大于 64 时，会自动转成红黑树，查找效率从 O(n) 降到 O(log n)。

2）开放寻址法（Open Addressing）
发生冲突时，按某种探测策略在数组中找下一个空位存放。线性探测、二次探测都属于这类。 `ThreadLocalMap` 用的就是线性探测，但容易产生堆积问题。

```
// HashMap 中处理碰撞的核心逻辑片段
int hash = hash(key.hashCode());
int i = (n - 1) & hash; // 定位桶
if (tab[i] == null || (e = tab[i]) == null) {
    tab[i] = newNode(hash, key, value, null);
} else {
    // 碰撞了，遍历链表或树插入
}
```

一般来说，链地址法更灵活，适合冲突较多的场景；开放寻址法内存紧凑，但负载高时性能下降明显。选哪种得看具体需求和数据特征。

Java 中有哪些集合类？请简单介绍

Java 的集合类主要分两大派，一个是 Collection 接口下的各种实现，另一个是 Map 系的键值对结构。面试里聊到这块，重点其实是搞清楚每种集合适用的场景和底层机制。

Collection 里最常用的是 List、Set 和 Queue。
List 是有序可重复的，ArrayList 内部用数组实现，随机访问快，但中间插入删除得搬数据，性能损耗大；LinkedList 是双向链表，适合频繁增删的场景，尤其是头尾操作。
Set 不允许重复元素，HashSet 基于 HashMap 实现，查找添加都是 O(1)，但不保序；LinkedHashSet 能维持插入顺序；TreeSet 则能自然排序，底层是红黑树。

Queue 一般用在任务调度，比如 ArrayDeque 是数组实现的双端队列，ConcurrentLinkedQueue 是无锁线程安全队列，DelayQueue 适合定时任务。

Map 才是脏活累活的主力。HashMap 最常见，数组 + 链表/红黑树结构，非线程安全，但性能好；Hashtable 就是老古董了，方法全 synchronized，基本被 ConcurrentHashMap 取代，后者用了分段锁 + CAS，在高并发下能扛住压力。
LinkedHashMap 能记住插入顺序，适合做 LRU 缓存；TreeMap 支持按键排序。

1）ArrayList 查找快，增删慢
2）LinkedList 增删快，查找慢
3）HashMap 性能高，并发用 ConcurrentHashMap
4）需要排序考虑 TreeSet / TreeMap
5）讲究插入顺序用 LinkedHashMap

Java 中的 WeakHashMap 是什么？

WeakHashMap 的关键在于它对内存的敏感处理。它的 key 是弱引用，只要 GC 扫到了，不管内存够不够，这个 key 就会被回收。

这意味着，哪怕你还在用 value，只要 key 被回收了，整个 entry 就失效了。所以它适合做缓存，但不是那种要求强一致的缓存。

典型场景是监听器注册表，比如 Swing 里的事件监听。注册的时候把 listener 当 key 放进去，一旦外面没人引用它，WeakHashMap 自动就清理了，避免内存泄漏。

对比 HashMap，它不会自动清理；而 WeakHashMap 在 key 回收后，下次 get 或 size 的时候会触发 expungeStaleEntries，把陈旧的 entry 清掉。

代码上其实用法差不多：

```
Map<SomeObject, String> map = new WeakHashMap<>();
SomeObject key = new SomeObject();
map.put(key, "data");
key = null; // 失去强引用
System.gc(); // 极端情况下触发
// 下次访问 map，可能就拿不到数据了
```

1）不要指望 WeakHashMap 存长期数据
2）别把它当普通 Map 用，尤其是 key 会被频繁丢弃的场景

3）GC 触发时机不确定，行为有延迟性

说说 Java 中 HashMap 的原理？

HashMap 本质是个数组加链表（或红黑树）的结构，存数据靠 key 的 hash 值定位桶位置。数组初始长度是 16，负载因子默认 0.75，到 12 个元素就扩容，避免冲突太多影响性能。

key 的 hash 值不是直接用，而是再做一次扰动运算，把高位也参与映射，减少碰撞概率。定位桶用 `(n - 1) & hash` 而不是取模，位运算更快。

1）数组里每个桶刚开始是 Node 节点，链表长度超过 8 且数组长度大于等于 64，链表转成红黑树，防止极端情况下查询退化成 O(n)
2）如果长度小于 6，即使 hash 冲突严重也先不转树，因为数组太小扩容更划算
3）删除节点时树长度小于 6 又会转回链表

put 操作流程：算 hash → 找桶 → 空则新建，否则遍历链/树插入 → 判断是否扩容。get 就反过来查。

注意多线程下 put 可能导致环形链表，所以并发场景得用 ConcurrentHashMap。JDK 8 虽然优化了扩容时的头插改尾插，但依然不能保证线程安全。

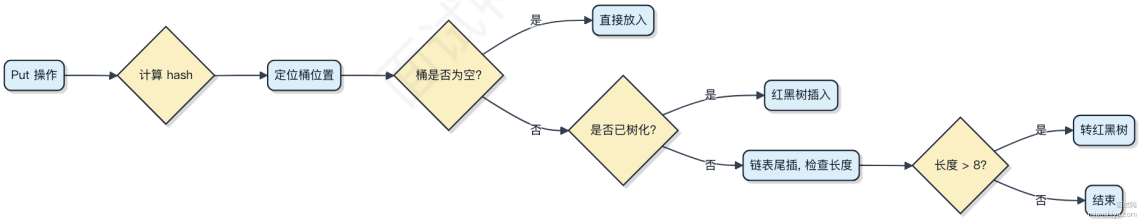
使用 HashMap 时，有哪些提升性能的技巧？

1）初始化时给足容量，避免频繁扩容。HashMap 扩容成本高，每次都要重新计算 hash 并搬数据。如果知道大概要存 100 个元素，别用默认的 16 初始容量，直接设成 `new HashMap<>(128)`，负载因子保持 0.75 就行。

2）自定义 key 类型时，**hashCode** 和 equals 一定要写好。hash 冲突多了会退化成链表甚至红黑树，查找从 O(1) 变成 O(logn)。比如用字符串做 key 没问题，JDK 已优化；但自定义对象必须重写这两个方法，不然不同实例永远不相等，还可能产生大量冲突。

3）极端情况下，单个桶里节点超过 8 个才会转红黑树。但如果 key 的 hash 值分布不均，比如都挤在几个桶里，那这些桶的查找效率立马下降。所以设计 key 的时候尽量让 hash 均匀分布。

4）并发场景别用 HashMap，它不是线程安全的。多个线程同时 put 可能导致链表成环或者数据丢失。真要并发读写，上 ConcurrentHashMap，JDK 8 后用了 synchronized + CAS 分段锁优化，性能比老版本还好。



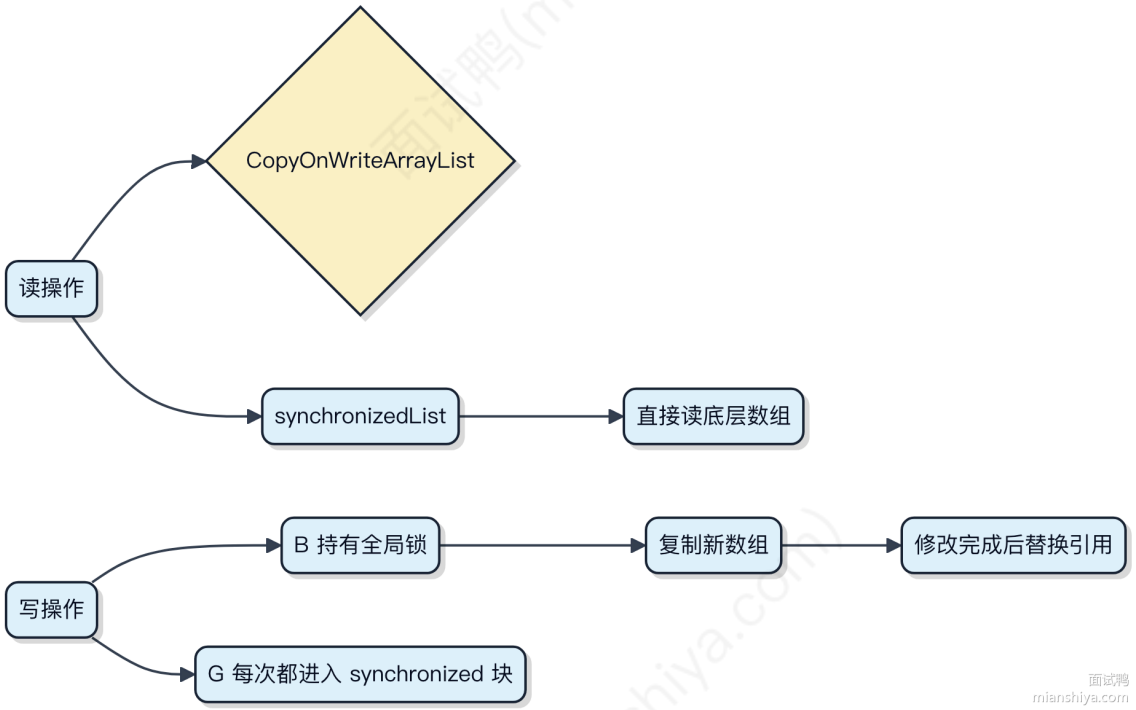
Java 的 CopyOnWriteArrayList 和 Collections.synchronizedList 有什么区别？分别有什么优缺点？

CopyOnWriteArrayList 和 synchronizedList 都是线程安全的 List 实现，但它们的设计哲学和适用场景完全不同。

CopyOnWriteArrayList 的核心是**写时复制**。每次修改操作（add、set、remove）都会创建一个新的数组副本，读操作则完全无锁。这就意味着读读、读写不互斥，但写写会竞争锁。适合读多写少的场景，比如监听器列表、观察者列表，像 Dubbo 的服务发现通知列表就用到了它。缺点也很明显，写操作成本高，而且可能内存占用翻倍，实时性差——刚写完不一定马上被别的线程读到。

synchronizedList 是基于装饰器模式，把一个普通 List 包装起来，所有方法都加上 synchronized 锁。读写都得抢同一把锁，吞吐量低。虽然简单直接，但在高并发读场景下，性能远不如 CopyOnWriteArrayList。不过它内存开销小，数据强一致性好，适合写操作频繁、对内存敏感的场景。

1）CopyOnWriteArrayList 读操作不加锁，适合读远多于写的场景
2）synchronizedList 所有操作都加锁，适合兼容老代码或写操作较多的情况
3）CopyOnWriteArrayList 有**内存膨胀**和**数据延迟可见**问题，使用时要警惕



你遇到过 ConcurrentModificationException 错误吗？它是如何产生的？

遍历集合时如果被其他线程或当前线程修改了结构，就会抛 ConcurrentModificationException。这个异常不是线程安全问题的唯一表现，但它是个明确的信号：**fail-fast 机制**被触发了。

Java 里像 ArrayList、HashMap 这些非线程安全集合，默认都有快速失败机制。它们内部维护一个 `modCount`，记录结构被修改的次数。遍历时会拿这个值和期望值比对，一旦不一致就抛异常。

比如用 for-each 循环删元素，看似单线程，也会中招：

```
for (String s : list) {
    if ("toRemove".equals(s)) {
        list.remove(s); // 直接抛 ConcurrentModificationException
    }
}
```

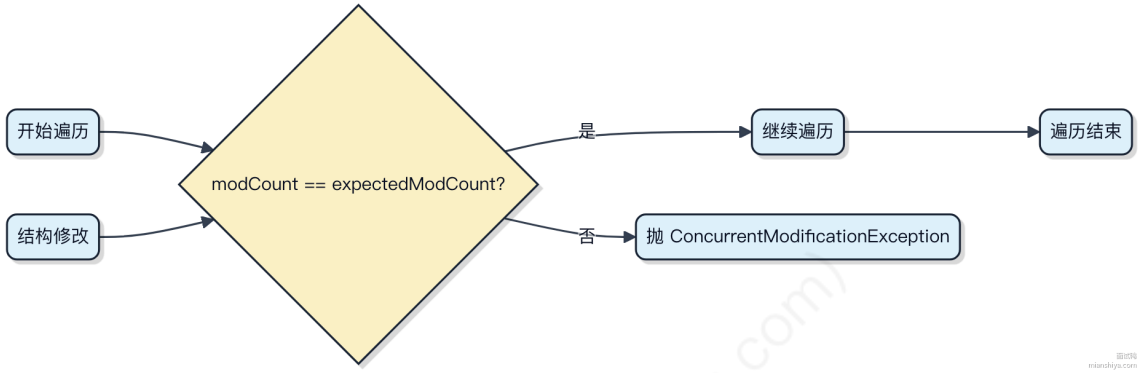
想安全删除，得用迭代器自己的 remove 方法：

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if ("toRemove".equals(s)) {
        it.remove(); // 正确姿势，内部会同步 modCount
    }
}
```

多线程场景更常见。比如一个线程读，另一个写 HashMap，哪怕只是 put，大概率也会触发异常。这时候要么用 ConcurrentHashMap，要么加锁。

ConcurrentHashMap 就不会抛这个异常，因为它用了更细粒度的同步策略，读操作不阻塞，**结构修改**也能保证一致性，压根不依赖 fail-fast 机制。

1）单线程遍历+修改，别直接调集合的增删方法
2）多线程场景优先上并发容器，比如 CopyOnWriteArrayList 或 ConcurrentHashMap
3）实在要用非线程安全集合，就得自己包 synchronized，但性能差很多



Java 中的 CopyOnWriteArrayList 是什么？

CopyOnWriteArrayList 是个典型的读写分离数据结构，主要用在**读多写少**的并发场景。它的思路很直接：读操作完全不加锁，写操作则通过复制底层数组来完成，保证线程安全。

读操作比如 `get` 、迭代器遍历，直接访问内部数组，压根不经过任何锁或 CAS，性能极高。像 Dubbo 的服务发现列表、ZooKeeper 客户端的监听器注册，就常用它存观察者列表，毕竟通知是高频读，注册却很少发生。

写操作比如 `add` 、 `set` 、 `remove` ，会先拿到独占锁，然后拷贝一份新数组，在新数组上修改，最后替换引用。整个过程老数组一直可用，所以读操作不会受影响。但代价也很明显，每次写都涉及数组复制，如果列表很大，比如上万元素，写一次可能就搞不定。

1）写操作开销大，不适合频繁写或大数据集
2）迭代器创建时就固定了内容，不支持 fail-fast，不会抛 ConcurrentModificationException
3）适合场景：事件监听器列表、配置缓存、路由表等读远多于写的场合

代码上看核心逻辑：

```java
public boolean add(E e) {
    synchronized (lock) {
        Object[] elements = getArray();
        int len = elements.length;
        Object[] newElements = Arrays.copyOf(elements, len + 1);
        newElements[len] = e;
        setArray(newElements);
    }
    return true;
}
```

写操作的锁只用来保护新数组的构建和替换，时间窗口小，但依然比普通 ArrayList 慢一个量级。

为什么 Java 的 ConcurrentHashMap 不支持 key 或 value 为 null?

ConcurrentHashMap 不让 key 或 value 为 null，根本原因是为了**避免多线程下的歧义问题**。

考虑这个场景：两个线程同时读一个 key，一个返回 null，你没法判断这个 null 是代表“没找到这个 key”，还是“找到了但 value 就是 null”。在并发环境下，这种不确定性会直接搞垮程序逻辑。HashMap 单线程下能存 null，是因为你可以用 `containsKey` 先判断，但在并发容器里，`containsKey` 和 `get` 之间可能已经被其他线程改了，这招就不灵了。

所以 ConcurrentHashMap 索性一刀切，**key 和 value 都不让为 null**，从源头上杜绝歧义。这不是设计缺陷，而是权衡后的安全选择。

拿实际项目来说，像用 ConcurrentHashMap 做本地缓存时，如果查不到就该走后端存储，而不是靠 null 来判断缺失。如果你真遇到 null 值需求，建议用 Optional.ofNullable 包一层，这样语义清晰又安全。

代码层面，它会在入口直接抛空指针：

```java
if (key == null || value == null)
    throw new NullPointerException();
```

整个设计思路就是：牺牲一点灵活性，换并发下的确定性和安全性。这在高并发系统里，比如用在 Dubbo 的本地缓存或 Netty 的状态管理中，是非常值得的。

Java 中 ConcurrentHashMap 的 get 方法是否需要加锁?

ConcurrentHashMap 的 get 方法不需要加锁，这是它和 Hashtable 最关键的区别之一。

JDK 8 之后的实现里，get 操作完全无锁，靠的是 **volatile** 读和**数组元素的 volatile 语义**来保证可见性。Node 数组的每个桶（bucket）里的节点，其 val 和 next 指针都是用 volatile 修饰的，这样在读的时候能直接看到最新值，压根不经过锁。

1）查找时先定位到桶，然后遍历链表或红黑树 2）每一步读取 node.val 或 node.next 都是 volatile 读，不会被重排序，也不会读到脏数据 3）整个过程没有 synchronized 或 ReentrantLock 参与

当然前提是：size 或 isEmpty 这种聚合操作不保证实时性，因为可能在计算时 map 正在并发修改。

对比来看，Hashtable 的 get 是全程 synchronized 的，吞吐量差很多。而 ConcurrentHashMap 在大多数读多写少场景下，比如缓存、配置中心本地副本，都能扛住高并发读。

代码上看，核心就是这几句：

```
transient volatile Node<K,V>[] table;
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    volatile V val;        // 注意是 volatile
    volatile Node<K,V> next; // 也是 volatile
}
```

Java 中 ConcurrentHashMap 1.7 和 1.8 之间有哪些区别？

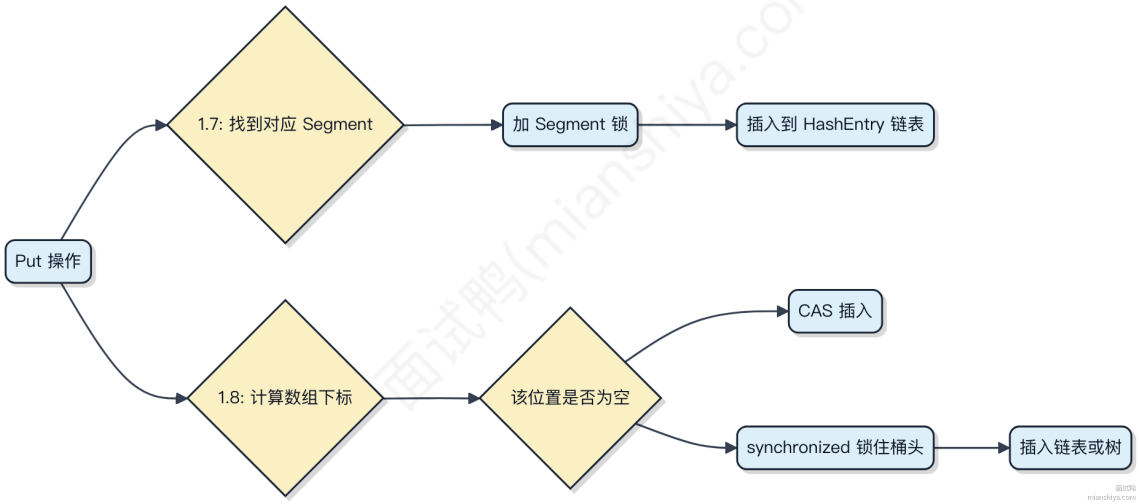
ConcurrentHashMap 的演进其实是个典型的“从锁竞争到 CAS + synchronized 优化”的过程，1.7 到 1.8 的改动几乎是重写级别。

1.7 用的是分段锁 Segment，每个 Segment 是一个 ReentrantLock 保护的小哈希表。默认 16 个 Segment，意味着最多 16 个线程能并发写。一旦线程数超过这个数，后面的就只能阻塞等待锁。Segment 继承自 ReentrantLock，结构复杂，内存占用也高。

到了 1.8，直接抛弃了 Segment，改用 **CAS + synchronized** 控制并发。Node 数组加链表/红黑树，写操作对数组桶位加 synchronized 锁，粒度更细。读操作完全无锁，靠 volatile 保证可见性。扩容时还能支持并发迁移，由多个线程一起搬数据。

另一个关键变化是哈希冲突的处理。1.7 是纯链表，容易在极端情况下退化成 O(n)。1.8 引入了链表转红黑树机制，当链表长度超过 8 且数组长度大于 64，就转换成红黑树，查找性能从 O(n) 提升到 O(log n)，抗攻击能力更强。

扩容效率也提升了。1.8 用了“多线程协助扩容”机制，一个线程发现需要扩容，可以只负责搬一部分桶，其他线程写数据时也会顺手帮忙搬，真正做到并发扩容。



Java 中的 IdentityHashMap 是什么？

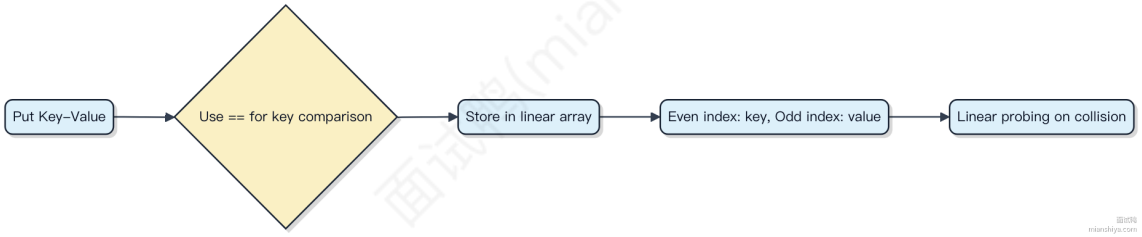
`IdentityHashMap` 不按 `equals` 比较键，而是用 `==` 判断 key 是否相等。也就是说，只有当两个引用指向同一个对象时，才认为 key 相同。

1）它不遵循 `Map` 接口的通用约定，比如和 `HashMap` 行为就不一样。你 put 一个 key，必须用完全相同的引用 get 才能拿到值，哪怕两个对象逻辑上 equals 为 true 也没用。

2）底层不是用红黑树或拉链法处理哈希冲突，而是采用**线性探测法**开放寻址，数组长度总是 2 的幂次。它的 Entry 直接存放在 Object 数组里，奇偶位分别放 key 和 value，节省空间也提升缓存命中率。

3）性能上比 `HashMap` 略快，因为跳过了 `hashCode()` 和 `equals()` 调用，适合做代理对象映射、AOP 中的目标对象追踪这类场景。Spring 的某些内部实现就用它来快速关联实例对。

注意别在常规业务里用它替代 `HashMap`，否则容易出现“明明 key 一样却取不到值”的问题。它的设计目标很明确：基于引用一致性做映射。



代码长这样：

```
IdentityHashMap<Key, Value> map = new IdentityHashMap<>();
Key k1 = new Key();
Key k2 = new Key();
map.put(k1, "first");
map.put(k2, "second");
// 即便 k1.equals(k2) 为 true，k1 和 k2 仍是不同 key
```

Java 中的 TreeMap 是什么？

TreeMap 是 Java 里基于红黑树实现的有序 Map，存进去的 key 会自动排序。遍历它的 keySet 或 entrySet 时，拿到的结果是按 key 升序排列的，不像 HashMap 那样无序。

1）底层数据结构是红黑树，一种自平衡二叉查找树。插入、删除、查找的时间复杂度都是 O(log n)，性能稳定，不会像普通二叉搜索树那样退化成链表。

2）它实现了 SortedMap 和 NavigableMap 接口，能干不少事。比如找小于某个 key 的最大键 `lowerKey()`，找大于等于某个 key 的最小键 `ceilingKey()`，还能截取一段范围 `subMap()`。这些在做区间查询时特别有用，像用时间戳做 key 查某段时间内的记录。

3）key 必须具备可比性。要么 key 类型实现 Comparable 接口，要么构造时传入 Comparator。不然运行时直接抛异常。

代码上长这样：

```
TreeMap<String, Integer> map = new TreeMap<>();
map.put("b", 2);
map.put("a", 1);
map.put("c", 3);
System.out.println(map.firstKey()); // 输出 a
```

注意点：如果只是想存键值对，不要求顺序，优先用 HashMap，O(1) 操作更快。TreeMap 主要用在需要有序访问的场景，比如用 ZSET 做排行榜时，后端临时排序可以用它，但量大了还是得上 Redis。

Java 中的 LinkedHashMap 是什么？

LinkedHashMap 是 HashMap 的子类，它在哈希表的基础上，**额外维护了一条双向链表**，用来记录插入顺序或者访问顺序。

这就让它能按顺序遍历 key-value 对。默认情况下，它是按插入顺序排列的。比如你 put("a", 1)，再 put("b", 2)，遍历时先 a 后 b。如果你构造时开启 accessOrder 模式，就会变成按访问顺序排列，最近访问的放到最后，适合实现 LRU 缓存。

像 MyBatis 的缓存、Spring MVC 处理请求参数绑定，都用到了 LinkedHashMap 来保证顺序性。它和 TreeMap 不一样，TreeMap 是靠 Comparable 或 Comparator 排序，底层是红黑树，性能开销更大。

代码上基本和 HashMap 一样用：

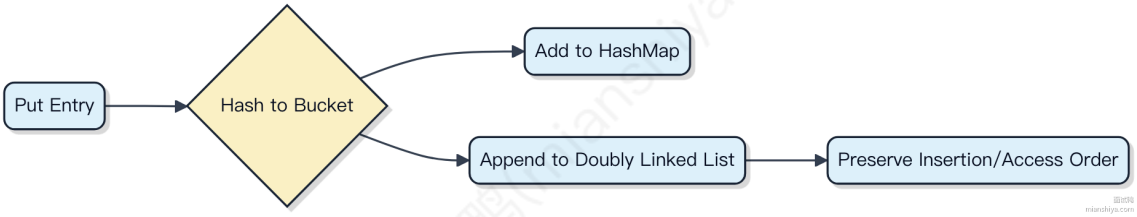
```
Map<String, Integer> map = new LinkedHashMap<>();
map.put("first", 1);
map.put("second", 2);
```

遍历结果就是插入顺序。如果想实现 LRU，可以重写 removeEldestEntry 方法：

```
protected boolean removeEldestEntry(Map.Entry eldest) {
    return size() > MAX_SIZE;
}
```

这时候超过容量会自动淘汰最老的数据。注意初始容量和负载因子别设得太小，不然频繁扩容影响性能。

它的缺点是内存占用比 HashMap 稍高，因为要存前后指针。但大多数场景下这点开销不值一提。



JDK 1.8 对 HashMap 除了红黑树还进行了哪些改动？

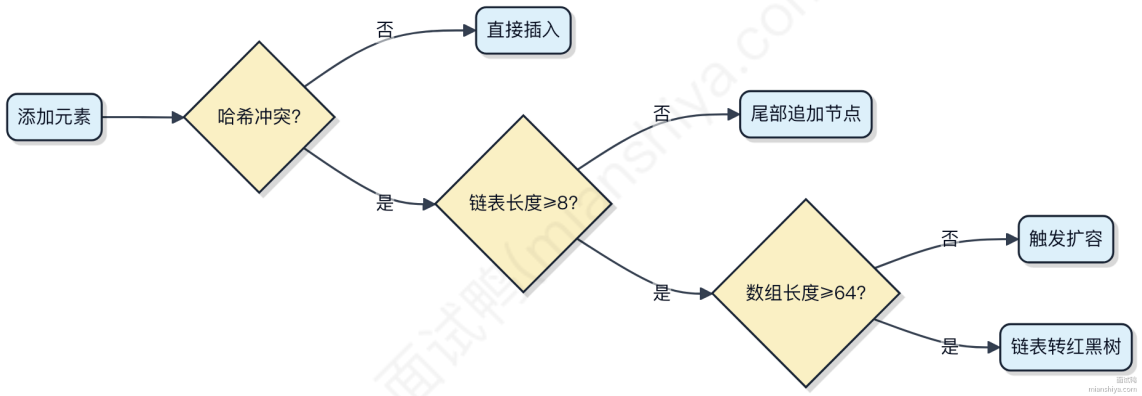
JDK 1.8 对 HashMap 的优化不止是引入红黑树这么简单，还有几个关键调整让它的性能更稳。

链表转红黑树的条件是 **bin 中节点数 ≥ 8 且 数组长度 ≥ 64**。如果只是链表长但数组太小，会优先扩容，而不是直接建树。这样避免在哈希冲突还没真正严重时就引入复杂的树结构，毕竟树的开销比链表大。

另一个容易被忽略的点是 hash 计算方式。JDK 1.8 仍然用 `(n - 1) & hash` 定位桶，但对 key 的 hashCode 做了**二次扰动**：先异或低16位和高16位，减少低位碰撞概率。这招在 key 的 hashCode 分布不均时特别有用，比如只递增的 Integer。

扩容机制也有变化。老版本是头插法，多线程下可能形成环，导致死循环。1.8 改成尾插法，迁移时节点顺序不变，彻底避开这个问题。而且扩容后的新位置要么在原索引，要么在原索引 + oldCap 位置，计算也快。

代码上看插入逻辑，核心还是 `putVal` 方法，但处理树化和拆分树的分支明显变多了。尤其是 `treeifyBin` 里会先看数组大小，不够就不树化。



这些改动合起来，让 HashMap 在高冲突场景下从 O(n) 降到 O(log n)，同时日常使用也没增加额外负担。

为什么 JDK 1.8 对 HashMap 进行了红黑树的改动？

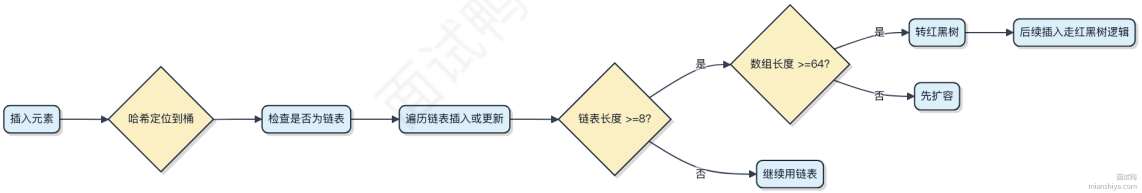
JDK 1.8 对 HashMap 的优化，主要解决哈希冲突严重时的性能退化问题。当大量元素落在同一个桶时，链表会变得很长，查找从 O(1) 退化到 O(n)。为控制最坏情况的时间复杂度，引入了红黑树机制。

具体触发条件是：某个桶内的节点数达到 8，并且当前数组长度大于等于 64，链表就会转成红黑树。这样最坏查找性能能稳定在 O(log n)。反过来，如果红黑树节点减少到 6，又会退化回链表，避免维护红黑树的额外开销。

这个设计其实是空间和时间的权衡。链表占用小，但查询慢；红黑树查询快，结构更复杂。阈值选 8 和 6 是为了防止频繁来回转换，用了泊松分布做依据，碰撞达到 8 的概率已经极低。

常见误区是认为只要 put 就可能转红黑树，其实前提是**扩容也解决不了散列问题**。比如所有 key 都只落在少数几个桶里，扩容后还是冲突，这时候链表长度才会持续增长。

代码上你不需要感知这个过程，HashMap 内部自动切换。但如果你的 key 没有合理实现 hashCode()，比如总返回 1，那所有元素都挤在一个桶，性能立马崩成 O(n)。



为什么 Java 中 HashMap 的默认负载因子是 0.75？

负载因子（load factor）决定了 HashMap 什么时候扩容。默认值 0.75 是空间和时间成本之间的一个权衡结果。

1）如果负载因子太小，比如 0.5，那数组还没装满就触发扩容，内存利用率低，但冲突少，查找快。
2）如果负载因子太大，比如 0.9，内存用得更满，但哈希冲突概率显著上升，链表变长，查找性能下降。

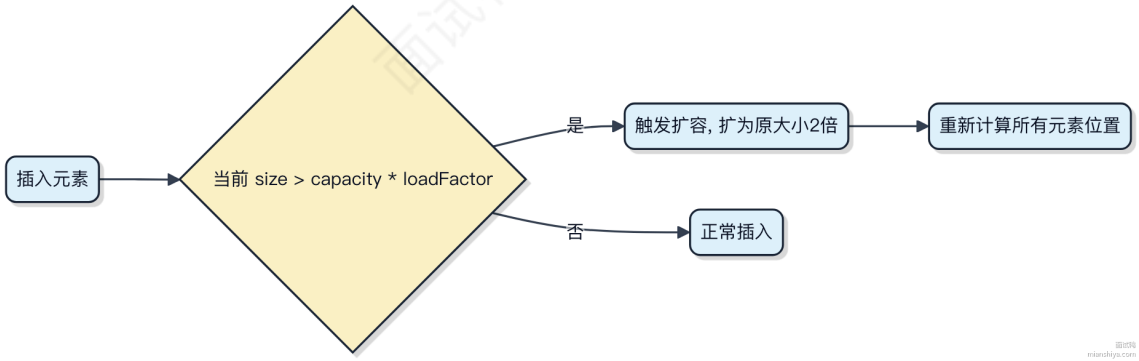
HashMap 的设计目标是在**平均情况下**实现 O(1) 的存取效率。0.75 这个值能让哈希桶的填充程度适中，在不浪费太多空间的前提下，把冲突控制在可接受范围。

JDK 源码注释里也提过，基于泊松分布统计，当负载因子为 0.75 时，一个桶里链长度超过 8 的概率极低，这和红黑树转换阈值的设计也能对上。

代码上看，初始化时如果不指定，就是 0.75：

```
public HashMap() {
    this.loadFactor = 0.75f;
}
```

实际开发中，如果你知道数据量会很大且稳定，可以提前算好容量，避免频繁 rehash。比如要存 1000 个元素，按 0.75 算，初始容量至少设成 `1000 / 0.75 ≈ 1333` ，再向上取最接近的 2 的幂，也就是 2048。



为什么 HashMap 在 Java 中扩容时采用 2 的 n 次方倍？

HashMap 的容量设计成 2 的 n 次方，主要是为了**快速计算元素的存储位置**。数组下标是通过 `hash & (capacity - 1)` 算出来的，这个位运算只有在容量为 2^n 时才等价于取模 `%` 。

如果容量是普通数字，就得用取模运算 `hash % capacity` ，这比位运算慢得多。而当 capacity 是 16、32、64 这种数时，capacity - 1 就是 15（0b1111）、31（0b11111）这种低位全 1 的二进制数，& 运算天然就能截断高位，实现均匀分布。

扩容翻倍也保证了这个特性一直成立。比如从 16 扩到 32，还是 2 的幂，下标计算方式不变，迁移数据时也能用 `hash & oldCapacity` 判断是否需要移动。

还有一个隐藏好处：JDK 8 的树化阈值判断和红黑树拆分逻辑，都依赖这个均匀分布特性来保证性能稳定。要是随便设个 100 当初始容量，不仅取模慢，还容易导致链表过长，查询退化成 O(n)。

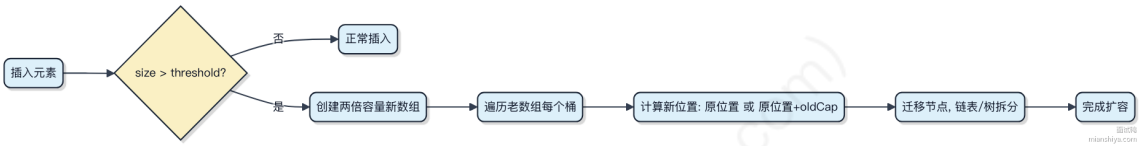
1）必须是 2 的幂才能用 & 替代 % 2）扩容翻倍维持了该性质连续性 3）配合扰动函数减少哈希冲突

Java 中 HashMap 的扩容机制是怎样的？

HashMap 扩容发生在元素数量超过阈值（threshold）时，这个阈值是容量（capacity）乘以加载因子（loadFactor），默认是 0.75。比如初始容量 16，阈值就是 16 * 0.75 = 12。一旦元素数超过 12，就会触发扩容。

扩容不是简单地把数组扩大一倍，而是创建一个新数组，长度翻倍，然后把老数据全部 rehash 搬到新桶里。这里有个优化点：因为容量总是 2 的幂，所以元素在新数组中的位置要么是原位置，要么是原位置加旧容量。JDK 8 利用这一点，通过位运算直接判断，避免重复计算 hash。

链表节点在迁移时会根据 `hash & oldCap` 的结果决定去向，等于 0 的留在原索引，不等于 0 的移到原索引 + 旧容量的位置。红黑树节点也类似，但会先拆分树，再判断是否需要转回链表。



注意扩容是**懒触发**的，只在 put 后检查时发生。多线程下并发 put 可能导致数据覆盖或链表成环，所以 HashMap 不是线程安全的。如果担心性能，可以预设足够大的容量，减少扩容次数。 ConcurrentHashMap 在这方面做了优化，扩容时支持并发读写，用的是**渐进式 rehash**。

Java 中的 HashMap 和 Hashtable 有什么区别？

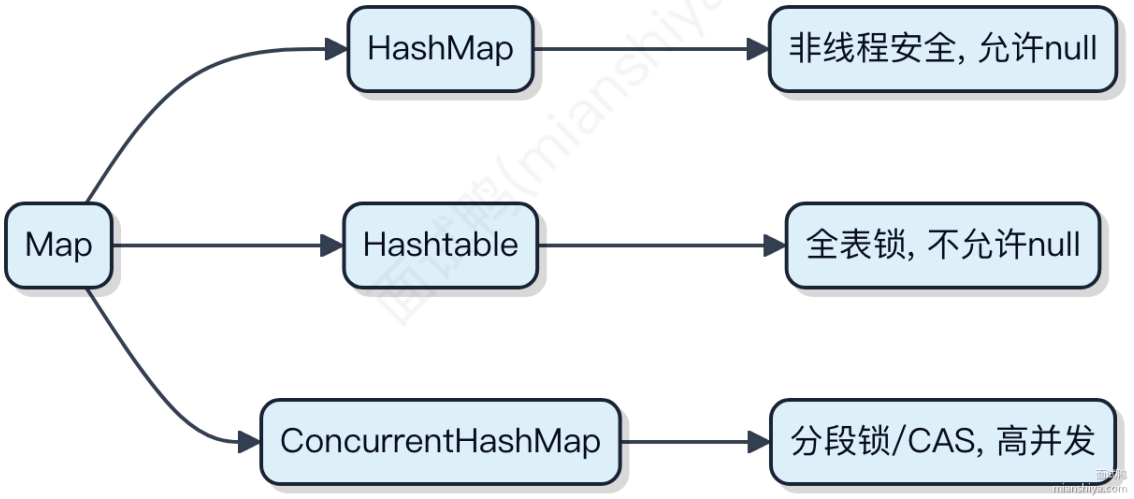
HashMap 允许 null 键和 null 值，而 Hashtable 不允许，一放就抛 NullPointerException。这个特性让 HashMap 在大多数场景下更灵活，比如做缓存时可以用 null 表示未命中。

线程安全方面，Hashtable 是**同步的**，每个方法都加了 synchronized，相当于把整个表锁住。虽然线程安全，但并发度低，同一时间只有 1 个线程能操作。HashMap 则完全不处理线程安全，适合单线程或外部加锁的场景。

如果要替代 Hashtable 的线程安全需求，现在一般用 ConcurrentHashMap。它采用分段锁（JDK 1.8 后是 CAS + synchronized）提升并发性能，在高并发下比 Hashtable 能扛住更多请求。

遍历方式也有差异。HashMap 使用 Iterator，支持 fail-fast 机制，一旦检测到并发修改会抛 ConcurrentModificationException。Hashtable 除了 Iterator，还支持 Enumeration，后者不抛异常，但可能看到过期数据。

1）底层结构相同，都是数组 + 链表/红黑树（JDK 1.8+）
2）初始容量和扩容机制类似，默认加载因子都是 0.75
3）HashMap 可以通过 Collections.synchronizedMap 包装成线程安全版本



实际开发中，Hashtable 基本属于历史遗留类，新项目直接上 HashMap 或 ConcurrentHashMap 就行。

数组和链表在 Java 中的区别是什么？

数组和链表是两种基础的数据结构，在 Java 里的表现和适用场景差异挺大，关键得看你怎么用。

1）内存布局上，数组在堆里申请一块连续的内存空间，大小固定，初始化时就得定下来。你声明一个 `int[1000]` ，JVM 就得一口气找 1000 个连续的 int 位置，中间不能断。链表不一样，每个节点 Node 分散在堆里，靠 next 指针连起来，增删节点就是改指针，不用找连续空间。

2）访问效率，数组支持随机访问， `get(i)` 是 O(1)，直接通过 base address + offset 算出位置。链表必须从头开始遍历， `get(i)` 最坏要 O(n)。

3）插入删除，数组中间插一个数，后面所有元素都得往后挪，代价是 O(n)。链表只要改前后节点的指针，O(1) 搞定，前提是已经定位到位置。所以像 LinkedList 在频繁增删的场景比 ArrayList 强。

```
// ArrayList 底层是数组
List<Integer> list = new ArrayList<>();
list.add(0, 1); // 后面元素全后移

// LinkedList 底层是双向链表
List<Integer> linked = new LinkedList<>();
linked.add(0, 1); // 只改指针
```

4）空间开销，数组只存数据，内存紧凑。链表每个节点得多存 1-2 个引用（next、prev），空间利用率低一些。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*